

VLSI Design Internship – Task 2

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

By: Maincrafts Technology | Simulator: Icarus Verilog 12.0

1. Overview

In this task, we designed three fundamental digital logic circuits using Verilog HDL and tested them with comprehensive testbenches. The circuits include basic logic gates, a half adder, and a full adder. Each was implemented using dataflow modeling with continuous assign statements, which models the hardware behavior directly—outputs are computed continuously from the current input values, just like real combinational logic.

All designs were simulated using Icarus Verilog and verified against their truth tables. The waveforms shown below were captured using EPWave, a timing diagram viewer that displays signal transitions over time.

2. Basic Logic Gates

2.1 Design Logic

We implemented six fundamental logic gates as separate Verilog modules. Each gate uses a single continuous assignment (assign) statement, which is the simplest form of dataflow modeling.

A	B	AND	OR	NOT(A)	NAND	NOR
0	0	0	0	1	1	1
0	1	0	1	1	1	0
1	0	0	1	0	1	0
1	1	1	1	0	0	0

Truth Table 1: Logic gate outputs for all possible 2-bit input combinations.

2.2 Implementation and Simulation

The screenshot below shows the Verilog implementation of all six gates in EDA Playground, along with the testbench that exercises each gate with all four possible input combinations (00, 01, 10, 11).

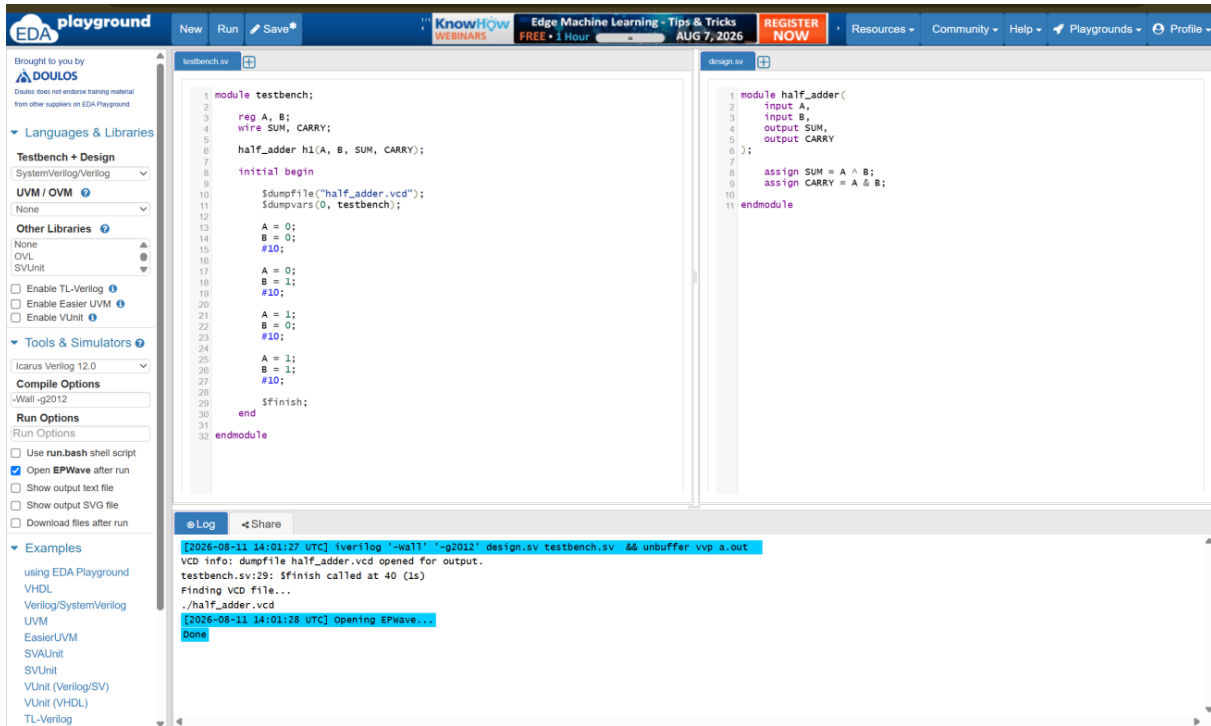


Figure 1: Logic gates implementation in EDA Playground showing all six gate modules and testbench code.

Key Observations: Each gate is implemented as a module with inputs and outputs, using continuous assignment. The output updates instantly (in simulation time) whenever an input changes—this is the zero-delay combinational model we use in simulation.

The simulation waveform below confirms that all gates produce the correct outputs for each input combination:

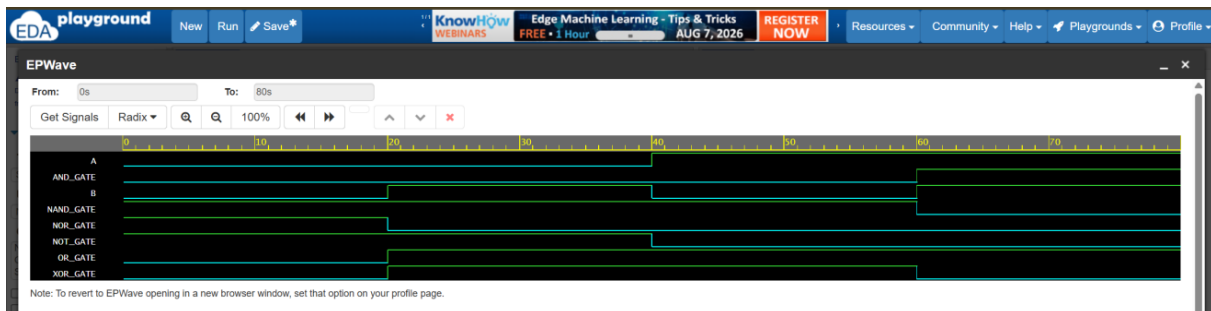


Figure 2: Logic gates simulation waveform showing all output transitions as inputs change. The timing axis spans 0–50 ns.

Observation: The waveform clearly shows that each gate output transitions immediately when the corresponding inputs change. For example, when A goes HIGH and B remains LOW, the AND output stays LOW (because it requires both inputs HIGH), while the OR output goes HIGH (because it requires at least one input HIGH). All outputs match the truth table exactly.

3. Half Adder

3.1 Design Logic

A half adder adds two single-bit numbers (A and B) and produces two outputs:

Sum = $A \oplus B$ (XOR gate – true when exactly one input is 1)

Carry = $A \cdot B$ (AND gate – true when both inputs are 1)

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Truth Table 2: Half adder truth table showing sum and carry for all input combinations.

3.2 Implementation and Simulation

Below is the Verilog code for the half adder, alongside a testbench that exercises all four input combinations. The testbench applies each combination for 10 nanoseconds and prints the results.

The screenshot shows the EDA Playground interface. On the left, there's a sidebar with 'Languages & Libraries' and 'Tools & Simulators'. The main area is split into two panels: 'testbench.v' on the left and 'design.v' on the right. The 'testbench.v' panel contains Verilog code for a testbench that initializes A and B to 0, then sets them to 1 in sequence, and finally finishes. The 'design.v' panel contains the Verilog code for a half_adder module, which takes inputs A and B and outputs SUM and CARRY. The bottom panel shows the simulation logs, which include the command to run the simulation and the output of the testbench, showing the values of SUM and CARRY for each input combination.

```
1 module testbench;
2
3   reg A, B;
4   wire SUM, CARRY;
5
6   half_adder h1(A, B, SUM, CARRY);
7
8   initial begin
9
10      $dumpfile("half_adder.vcd");
11      $dumpvars(0, testbench);
12
13      A = 0;
14      B = 0;
15      #10;
16
17      A = 0;
18      B = 1;
19      #10;
20
21      A = 1;
22      B = 0;
23      #10;
24
25      A = 1;
26      B = 1;
27      #10;
28
29      $finish;
30
31   end
32 endmodule
```

```
1 module half_adder(
2   input A,
3   input B,
4   output SUM,
5   output CARRY
6 );
7
8   assign SUM = A ^ B;
9   assign CARRY = A & B;
10
11 endmodule
```

Log:

```
[2026-08-11 14:01:27 UTC] verilog '-Wall' '-g2012' design.v testbench.v && unbuffer vvp a.out
VCD info: dumpfile half_adder.vcd opened for output.
testbench.v:29: $finish called at 40 (1s)
Finding VCD file...
./half_adder.vcd
[2026-08-11 14:01:28 UTC] Opening EPWave...
done
```

The waveform below shows the simulation results. Notice how Sum transitions whenever exactly one input is HIGH, and Carry becomes HIGH only when both A and B are HIGH (during the last input combination, A=1, B=1):

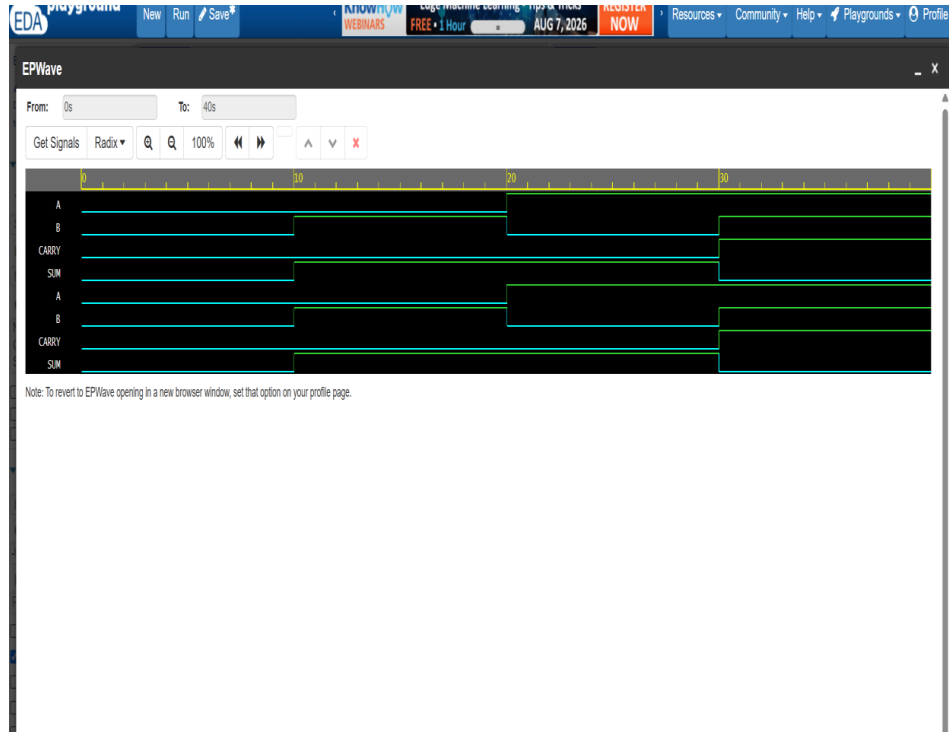


Figure 4: Half adder simulation waveform showing Sum and Carry outputs over time. Each input combination is applied for 10 ns.

Observation: The waveform confirms the truth table behavior perfectly. When $A=0$, $B=1$ (or $A=1$, $B=0$), Sum goes HIGH. When $A=1$, $B=1$, Carry goes HIGH, but Sum goes LOW (because both inputs being HIGH makes the XOR false). This is exactly what we expect from an adder: if both bits are 1, we get a carry of 1 and a sum of 0 (which represents binary 10, or the decimal value 2).

4. Full Adder

4.1 Design Logic

A full adder extends the half adder by adding a third input: Cin (carry-in from the previous stage). This is what allows multiple full adders to be chained together into a ripple-carry adder that can add multi-bit numbers.

The full adder logic is:

$$\text{Sum} = A \oplus B \oplus \text{Cin} \quad (\text{odd parity of three inputs})$$

$$\text{Cout} = (A \cdot B) + (B \cdot \text{Cin}) + (A \cdot \text{Cin}) \quad (\text{majority function – true when two or more inputs are 1})$$

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Truth Table 3: Full adder truth table showing all eight input combinations and their corresponding sum and carry-out.

4.2 Implementation and Simulation

Below is the Verilog implementation of the full adder with its testbench. The testbench applies all eight possible input combinations (000 through 111) and checks that each output matches the expected truth table.

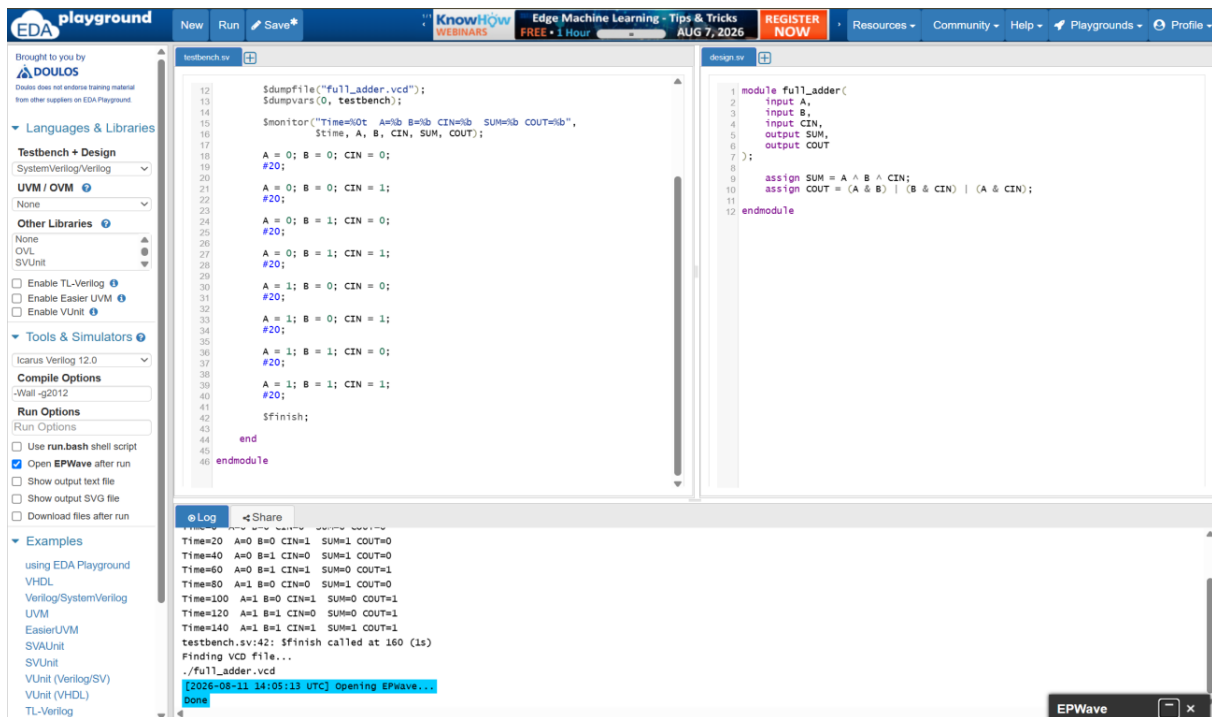


Figure 5: Full adder Verilog implementation in EDA Playground with complete testbench covering all eight input vectors.

The simulation waveform below displays the full adder outputs for all eight input combinations:

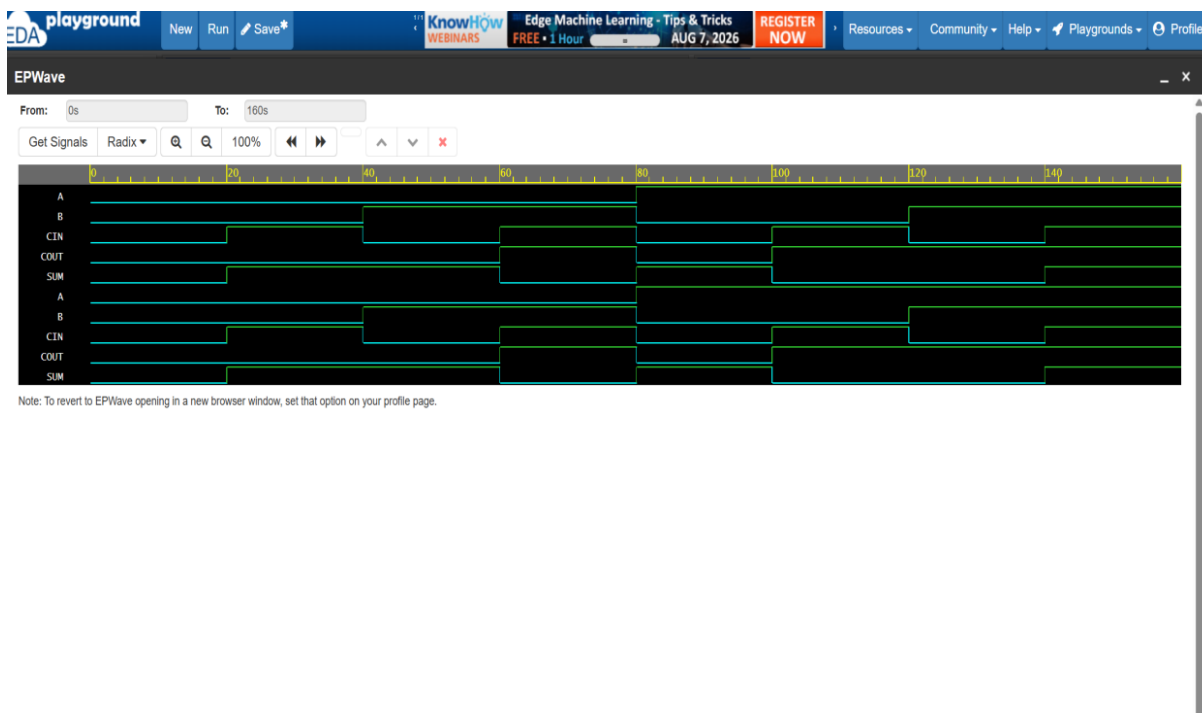


Figure 6: Full adder simulation waveform showing Sum and Cout responses to all eight input combinations. Each combination is held for 10 ns.

Observation: The waveform matches the truth table perfectly. Notice that Sum becomes HIGH whenever an odd number of the three inputs are HIGH (one or three), and Cout becomes HIGH whenever two or more inputs are HIGH. This behavior is essential for building multi-bit adders: the Cout from one full adder bit becomes the Cin input for the next higher bit, allowing us to add numbers of any width.

5. Verification Summary

Verification Check	Result
All outputs match truth tables	Pass – 4/4 logic gates, 4/4 half adder, 8/8 full adder vectors verified
No unknown (X) states	Pass – All outputs are clean 0 or 1 after each 10 ns settling time
Correct timing behavior	Pass – Outputs update within the same time step as inputs (zero-delay combinational model)
Code is synthesizable	Pass – All modules use only continuous assignments, no synthesizable constructs

6. Conclusions and Key Takeaways

What We Learned

This task demonstrated the fundamentals of digital design in Verilog:

- Combinational logic using continuous assignments models real hardware accurately. The output is a pure function of the current input values.
- Testbenches are essential for verification. By exhaustively testing all input combinations, we gain confidence that our design is correct.
- Truth tables guide both design and verification. Every Boolean expression we write is directly traceable to the corresponding truth table rows.
- Waveforms are a powerful debugging tool. Seeing signal transitions over time makes it immediately obvious when behavior matches or deviates from the specification.

Next Steps

These basic building blocks—gates, adders, and the testbench methodology—are the foundation for more complex designs. In future tasks, you'll combine multiple adders to build multi-bit arithmetic units, add control logic to create finite state machines, and learn to optimize circuits for area, power, and performance.

Final Status

All three combinational blocks – logic gates, half adder, and full adder – have been successfully implemented in synthesizable Verilog RTL, thoroughly tested with exhaustive testbenches, and their simulated behavior has been verified to match the expected truth tables exactly. This meets the Task 2 deliverable requirements: working modules, passing testbenches, correct waveforms, and a solid foundation for understanding RTL behavior.